# ПРАКТИЧНА РОБОТА №8

### Варіант 1

Я створила новий проєкт Next.js з використанням TypeScript. Потім створила сторінку для відображення публікацій. А для отримання даних використала API JSONPlaceholder:

https://jsonplaceholder.typicode.com/posts?_limit=8

Створила тип Post, який містить такі поля:

- id — номер публікації;
- title — заголовок;
- body — текст публікації.

За допомогою fetch() отримала 8 публікацій. Також додала перевірку response.ok, щоб обробляти помилки запиту. Далі створила файл loading.tsx для відображення повідомлення під час завантаження даних. Потім створила файл error.tsx для обробки помилок із кнопкою повторної спроби. І перевірила три режими кешування.

## 2. Режими кешування

#### 1. No-store

```
fetch("https://jsonplaceholder.typicode.com/posts?_limit=8", {
  cache: "no-store",
});
```

Дані завантажуються заново під час кожного запиту, без використання кешу.

```
export default async function Home() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts?_limit=8",
    { cache: "no-store" }
  );
```

#### 2. Force-cache

```
fetch("https://jsonplaceholder.typicode.com/posts?_limit=8", {
  cache: "force-cache",
});
```

Використовуються збережені в кеші дані, якщо вони доступні.

```
export default async function Home() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts?_limit=8",
    { cache: "force-cache" }
  );
```

**3. Revalidate**

fetch("https://jsonplaceholder.typicode.com/posts?_limit=8", {
  cache: "force-cache",
});
Дані кешуються, а повторна перевірка актуальності кешу виконується через 60 секунд.

```
export default async function Home() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/posts?_limit=8",
    { next: { revalidate: 60 } }
  );
```

### 3. Порівняння режимів кешування

| Режим | Особливості |
|---|---|
| no-store | Не використовує кеш, отримує нові дані |
| force-cache | Використовує кешовані дані |
| revalidate: 60 | Оновлює кеш після 60 секунд |

## 4. Висновок

Під час практичної роботи я навчилася отримувати дані із зовнішнього API за допомогою fetch(). Створила сторінку з публікаціями та додала стани завантаження й обробки помилок. Також ознайомилася з трьома режимами кешування: no-store, force-cache та revalidate. Зрозуміла, як кешування впливає на отримання та оновлення даних.

## 5. Контрольні запитання

**1. Для чого використовується fetch()?**

Для отримання даних із сервера або зовнішнього API.

**2. Навіщо потрібні async та await?**

async позначає асинхронну функцію, а await очікує завершення запиту.

**3. Для чого перевіряти response.ok?**

Щоб перевірити, чи успішно виконано HTTP-запит.

**4. Чим відрізняються force-cache та no-store?**

force-cache використовує кешовані дані, а no-store щоразу отримує нові дані.

**5. Що означає next: { revalidate: 60 }?**

Це означає, що кеш потрібно повторно перевіряти через 60 секунд.

**6. Для чого потрібні loading.tsx та error.tsx?**

loading.tsx показує повідомлення під час завантаження, а error.tsx відображає помилку та дозволяє повторити спробу.